DNA/RNA analysis pipeline

Created	@June 24, 2024 6:56 PM
Class	DNA/RNA mod 2
Type	Lab
Reviewed	☐

Pipeline steps

1. Load raw data with minfi and create an object called RGset storing the RGChannelSet object

   `minfi` is an R package that provides comprehensive tools for analyzing methylation data from Infinium methylation arrays. It facilitates reading raw data, performing quality control, preprocessing, and various downstream analyses. The `RGChannelSet` object in `minfi` is a data structure that stores raw signal intensities from the red and green channels of the Illumina methylation arrays, essential for assessing DNA methylation levels across the genome.

   a. **Load the required libraries:** the first step is that we ensure the `minfi` and `BiocManager` packages to be installed. If not, install them first.

   ```
   if (!requireNamespace("BiocManager", quietly = TRUE))
       install.packages("BiocManager")

   BiocManager::install("minfi")

   library(minfi)
   ```

   b. **Load the raw data and create the RGChannelSet object:** Use the `read.metharray.exp` function to read the IDAT files and create an RGChannelSet object.

   ```
   rm(list=ls())
   setwd("~/Desktop/project_DNA_RNA/")
   library(minfi)

   # Set the directory in which the raw data are stored and load the samplesheet
   using the function read.metharray.sheet
   baseDir <- ("~/Desktop/project_DNA_RNA/Input_data_report")
   targets <- read.metharray.sheet(baseDir)

   # Create an object of class RGChannelSet using the function read.metharray.ex
   p
   RGset <- read.metharray.exp(targets = targets)
   save(RGset,file="RGset.RData")
   ```

2. Create the dataframes Red and Green to store the red and green fluorescences respectively

   a. **Create the dataframes Red and Green to store the red and green fluorescences respectively: e**xtract the red and green channel data from the RGChannelSet object and convert them to dataframes.

   ```
   # We extract the Green and Red Channels using the functions getGreen and getR
   ed
   Red <- data.frame(getRed(RGset))
   dim(Red) # rows: probes, columns: samples
   ```

```
[1] 622399      8
head(Red)

Green <- data.frame(getGreen(RGset))
dim(Green)
[1] 622399      8

head(Green)
```

3. Fill the following table: what are the Red and Green fluorescences for the address assigned to you? **Optional**: check in the manifest file if the address corresponds to a Type I or a Type II probe and, in case of Type I probe, report its color.

   My address is: 18744490

   Problem: Get the Red and Green fluorescences for the address: **18744490**, and check in the manifest file if the address corresponds to a Type I or a Type II probe.

   Retrieving the Red and Green fluorescences and the chemistry of the probes related to the following address: **18744490**. To check for the type of probes it is necessary to look into the Manifest file, which has been downloaded at the following from the from the Illumina website (http://support.illumina.com/array/array_kits/infinium_humanmethylation450_beadchip_kit/downloads.html) and then cleaned up (control and rs probes have been removed).

   a. **Get probe info**: Get the Red and Green fluorescences for the address: **18744490**

   ```
   # Address of interest
   address <- "18744490"

   # Get Red and Green fluorescence values for the specified address
   probe_red <- Red[address, ]
   probe_green <- Green[address, ]
   ```

   b. **Check for the type of the probe**: to check whether the address corresponds to a Type I or a Type II probe, we need to look into the manifest file.

   ```
   # Load the cleaned manifest file to check the type of the probe

   load('Illumina450Manifest_clean.RData')
   # check whether it is of type I or II from column 'Infinium_Design_Type'
   Illumina450Manifest_clean[Illumina450Manifest_clean$AddressA_ID=="18744490",
   'Infinium_Design_Type']
   [1] II
   Levels: I II
   # extract all data related to sample, red fluorescence, green fluorescence an
   d type and color to fill the datable
   df_address <- data.frame(Sample=colnames(probe_green), Red_fluor=unlist(probe
   _red, use.names = FALSE ), Green_fluor=unlist(probe_green, use.names = FALSE
   ), Type = "II")
   df_address
   ```

Sample	Red fluor	Green fluor	Type	Color
X200400320115_R01C01	5646	11390	II	
X200400320115_R02C01	6121	13072	II	
X200400320115_R03C01	5784	14138	II	
X200400320115_R04C01	5815	13203	II	
X200400320115_R02C02	786	2963	II	
X200400320115_R03C02	1165	1954	II	
X200400320115_R04C02	6440	11222	II	
X200400320115_R05C02	527	840	II	

Example Output

The output will be a data frame with the following columns:

- **Sample**: Sample names.
- **Red_Fluor**: Red fluorescence values.
- **Green_Fluor**: Green fluorescence values.
- **Type**: Probe type (I or II).
- **Color**: Color channel for Type I probes (Red or Green), NA for Type II probes.
  - Type II probes use a single bead type for both methylated and unmethylated states.
  - They measure the intensities of the methylated and unmethylated states using the same color channel.
  - As a result, Type II probes do not require separate color information for the red or green channels because they are not differentiated by color.

Moreover, this kind of probes has a correspondence only for the address A and not for the address B.

```
Illumina450Manifest_clean[Illumina450Manifest_clean$AddressB_ID=="18744490", 'Infin
ium_Design_Type']
factor()
Levels: I II
>
```

4. Create the object MSet.raw

   The `MSet.raw` object in the `minfi` package is an object that represents the raw methylation data in a preprocessed format. This object is typically generated from an `RGChannelSet` object, which contains raw signal intensities from the red and green channels of the Illumina methylation arrays. The `MSet.raw` object retains the raw intensity values but organizes them in a way that is suitable for downstream analysis.

   ```
   # Convert the RGChannelSet object to an MSet object:
   # Use the preprocessRaw function to create the MSet.raw object from the RGChanne
   lSet.
   MSet.raw <- preprocessRaw(RGset)
   MSet.raw
   # Note that now the number of rows is 485512, exactly the number of probes accor
   ding to the Manifest!
   save(MSet.raw,file="MSet_raw.RData")
   ```

   The `preprocessRaw` function does the following

   - Background Correction: Adjusts the fluorescence intensities for background noise.

- No Normalization: Unlike some other preprocessing functions, preprocessRaw does not perform normalization of the data.
- Conversion to MethylSet: Converts the data to a MethylSet object, which is a more convenient format for downstream analysis.

The command `MSet.raw` will display a summary of the `MSet.raw` object. The `MethylSet` object contains several slots (components) including:

- **Assay Data**: Matrices of methylated (`Meth`) and unmethylated (`Unmeth`) signal intensities.
- **Phenotype Data**: Information about the samples (e.g., sample IDs, phenotypic traits).
- **Feature Data**: Information about the probes (e.g., probe IDs, genomic coordinates).
- **Experiment Data**: Metadata about the experiment.

5. Perform the following quality checks and provide a brief comment to each step:

- QCplot
- check the intensity of negative controls using minfi
- calculate detection pValues; for each sample, how many probes have a detection p-value higher than the threshold assigned to each student?

a. **Step 1: Generate QC Plots:** Quality control (QC) plots are useful to visualize the quality of the samples and identify any potential outliers.

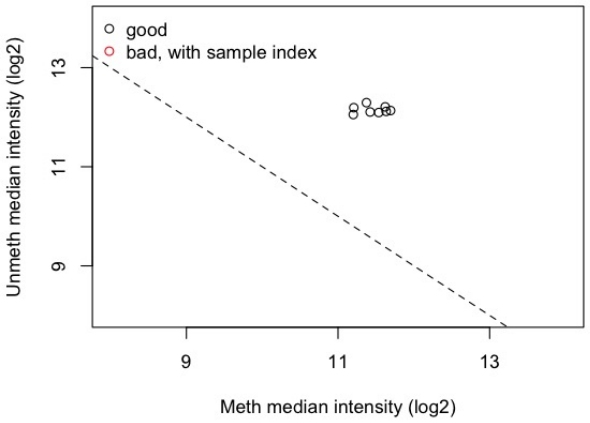
A QC plot based on the `getQC()` function focuses on the medians of methylation (Meth) and unmethylation (Unmeth) signal distributions across the entire chip. This plot serves as an initial assessment of data quality:

- **Interpreting Medians:** High median values for both Meth and Unmeth signals, plotted towards the right and upper parts of the QC plot, indicate data of good quality. Conversely, lower median values suggest lower data quality.
- **Limitations:**
  1. **Background Signal:** The plot does not consider background signal levels, which can affect overall data quality.
  2. **Sample Preparation Failures:** It also does not account for potential failures during sample preparation, which could impact signal integrity.

Using `plotQC()`, which utilizes the DataFrame generated by `getQC()`, provides a visual diagnostic tool to assess these median values and quickly identify samples or chips that may require further investigation or exclusion from downstream analysis.

This approach helps initial quality assessment but should be supplemented with additional checks for background noise and sample preparation issues for a more comprehensive evaluation of data quality.

```
qc <- getQC(MSet.raw)
qc
plotQC(qc)
```



In the QCplot is clear that all the samples have a good median for both methylated and unmethylated signals

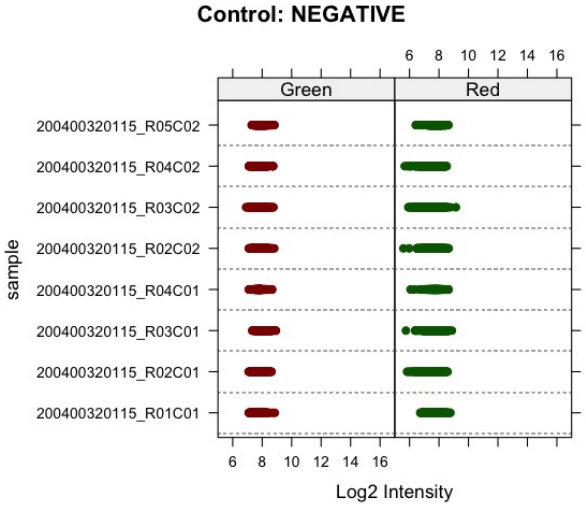
b. **Negative control intensity check**

Low and consistent intensities in negative controls indicate low background noise, ensuring the reliability of the observed signals.

Negative control probes are crucial for estimating background intensity in methylation array experiments. Typically, these probes should exhibit intensity values below 1000 units (log2(1000) = 10), indicating minimal background signal.

To assess these intensities using `controlStripPlot()`, we visually inspect the distribution of negative control probe intensities across samples. Ideally, all intensities should be clustered between 100 to 1000 units Any deviations above this threshold may suggest elevated background noise or other experimental issues affecting data quality.

```
controlStripPlot(RGset, controls="NEGATIVE")
```



Here, in both channel the intensity values are below 10, thus they are in the acceptable range.

c. **Count Probes Exceeding Threshold:**

Detection p-values help assess the reliability of the measurements for each probe. A low p-value indicates that the signal from a probe is significantly different from the background noise, suggesting a

reliable measurement. Conversely, a high p-value indicates that the probe signal is not distinguishable from background noise, which questions the reliability of that probe's measurement.

By identifying probes with high detection p-values, we can perform quality control on our data. Samples or probes with many high p-values may indicate poor data quality.

Removing or flagging probes with high detection p-values ensures that downstream analyses (e.g., differential methylation analysis, clustering, etc.) are based on reliable data. This filtering improves the robustness and accuracy of the results, reducing the likelihood of false positives or misleading conclusions.

```
# Set the detection p-value threshold to determine which probes are considere
d reliably detected.
threshold <- 0.05

detP <- detectionP(RGset)
failed <- detP>0.05


num_failed = colSums(failed)


df_failed <- data.frame(Sample=colnames(probe_green), Failed_Position=num_fai
led,
                        perc_failed_probes=(means_of_columns <- colMeans(fail
ed))*100,
                        row.names = NULL)


df_failed
# Print the results
df_failed

                Sample Failed_Position perc_failed_probes
1 X200400320115_R01C01              45        0.009268566
2 X200400320115_R02C01              26        0.005355171
3 X200400320115_R03C01              28        0.005767108
4 X200400320115_R04C01              32        0.006590980
5 X200400320115_R02C02             190        0.039133945
6 X200400320115_R03C02             130        0.026775857
7 X200400320115_R04C02              17        0.003501458
8 X200400320115_R05C02             406        0.083623062
```

Sample	Failed Probes	Percentage of Failed Probes
X200400320115_R01C01	45	0.0093%
X200400320115_R02C01	26	0.0054%
X200400320115_R03C01	28	0.0058%
X200400320115_R04C01	32	0.0066%
X200400320115_R02C02	190	0.0391%
X200400320115_R03C02	130	0.0268%
X200400320115_R04C02	17	0.0035%
X200400320115_R05C02	406	0.0836%

- Usually, **bad sample** contains **more than 5%** of bad probes.
- Most of the samples have a very low percentage of failed probes, well below 1%. This indicates that the data quality is generally high for these samples.
- **Specific Samples**:
  - **X200400320115_R01C01, X200400320115_R02C01, X200400320115_R03C01, X200400320115_R04C01, and X200400320115_R04C02** have extremely low percentages of failed probes (less than 0.01%), suggesting these samples are of excellent quality.
  - **X200400320115_R02C02 and X200400320115_R03C02** have higher percentages of failed probes (around 0.04% and 0.03%, respectively). Although these values are still relatively low, they are higher compared to the other samples and might warrant a closer look to ensure they are of acceptable quality.
  - **X200400320115_R05C02** has the highest percentage of failed probes at 0.0836%. While this is still below 0.1%, it is noticeably higher than the other samples. This could indicate potential issues with this sample, such as higher background noise or technical problems during processing. It might be worthwhile to further investigate this sample to ensure it doesn't impact your downstream analyses.

None of the samples has a percentage of failed probes higher than 5%, thus we can retain them.

6. Calculate raw beta and M values and plot the densities of mean methylation values, dividing the samples in WT and MUT (*suggestion: subset the beta and M values matrixes in order to retain WT or MUT subjects and apply the function mean to the 2 subsets*). Do you see any difference between the two groups?

   The methylation value is a continuous number ranging from 0 to 1. We can use the fluorescence intensity to generate that number, useful for the statistical analysis.

   DNA methylation levels can be measured through:

   - **β-values**

     represent the proportion of methylation at a given CpG site and range from 0 (absent methylation) to 1(completely methylated).

     - β-values are intuitive and easy to interpret but can be heteroscedastic, especially at extreme values (near 0 or 1).

     $\beta = \frac{M}{M+U}$

   - **M-values**

     M-values are log-transformed ratios of methylated to unmethylated signal intensities.

     It is a continuous variable which can in principle take on any value on the real line

     $M = log_2 \frac{M}{U}$

   $Beta_i = \frac{2^{M_i}}{2^{M_i}+1}$ $\quad$ $M_i = log_2(\frac{Beta_i}{1-Beta_i})$

   These values are derived from the fluorescence intensities measured by methylation arrays.

   Usually you get Beta value data frame from GenomicRatioSet and then generate the M-values data frame using the log2 function.

   Let's proceed with the analysis by calculating raw beta and M values, and then plotting the densities of mean methylation values, dividing the samples into WT (wild type) and MUT (mutant) groups.

   a. **Calculate Raw Beta and M Values dividing the samples into WT and MUT**

      `basename` extracts only the file name from each path, removing the directory part

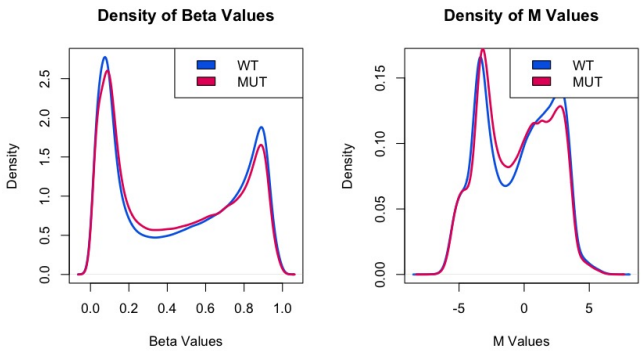
      ```
      wt <- basename(targets[targets$Group == "WT", "Basename"])
      mut <- basename(targets[targets$Group == "MUT", "Basename"])
      ```

```
wtSet <- MSet.raw[, colnames(MSet.raw) %in% wt]
mutSet <- MSet.raw[, colnames(MSet.raw) %in% mut]

wtBeta <- getBeta(wtSet)
wtM <- getM(wtSet)
mutBeta <- getBeta(mutSet)
mutM <- getM(mutSet)

mean_wtBeta <- apply(wtBeta, MARGIN=1, mean, na.rm=TRUE)
mean_mutBeta <- apply(mutBeta, MARGIN=1, mean, na.rm=TRUE)
mean_wtM <- apply(wtM, MARGIN=1, mean, na.rm=TRUE)
mean_mutM <- apply(mutM, MARGIN=1, mean, na.rm=TRUE)

d_mean_wtBeta <- density(mean_wtBeta, na.rm=TRUE)
d_mean_mutBeta <- density(mean_mutBeta, na.rm=TRUE)
d_mean_wtM <- density(mean_wtM, na.rm=TRUE)
d_mean_mutM <- density(mean_mutM, na.rm=TRUE)
```

b. **Plotting densities**

```
# Plot density of mean Beta and M values
par(mfrow=c(1,2))  # Set up the plotting area to have 1 row and 2 columns

# Plot density of mean Beta values
plot(d_mean_wtBeta, main="Density of Beta Values", col="#0067E6", lwd=2.5, xl
ab="Beta Values", ylab="Density")
lines(d_mean_mutBeta, col="#E50068", lwd=2.5)
legend('topright', legend=c("WT", "MUT"), fill=c("#0067E6", "#E50068"), cex=
1)

# Plot density of mean M values
plot(d_mean_wtM, main="Density of M Values", col="#0067E6", lwd=2.5, xlab="M
Values", ylab="Density")
lines(d_mean_mutM, col="#E50068", lwd=2.5)
legend('topright', legend=c("WT", "MUT"), fill=c("#0067E6", "#E50068"), cex=
1)

# Reset plotting layout
par(mfrow=c(1,1))
```

- The first `plot` function creates the density plot for mean Beta values, with `d_mean_wtBeta` in blue (`#0067E6`) and `d_mean_mutBeta` in red (`#E50068`).
- The second `plot` function creates the density plot for mean M values in the same color scheme.
- Legends are added to the top right of each plot to indicate which color corresponds to WT and MUT groups.



Overall, by looking at the two graphs seems that WT and MUT have a similar distribution for both the beta value and the M value. However, there are some differences: - *central values*: WT are slightly lower - *peaks values*: WT are slightly higher

- In the plot, both WT and MUT groups show a similar distribution pattern with peaks around 0.1 and 0.9, indicating bimodal distribution. This is typical for methylation data as many CpG sites are either fully unmethylated or fully methylated.

  The density curves for WT (blue) and MUT (red) are closely aligned, suggesting there is no significant difference in the overall distribution of methylation levels between the two groups.

- The M value density plot also shows a similar pattern for WT and MUT groups, with two main peaks, one around -5 and another around 5, which corresponds to the low and high Beta values respectively. Again, the density curves for WT and MUT are quite similar, indicating no major differences in methylation distributions between the groups.

- The similarity in the density distributions of both Beta and M values between WT and MUT groups suggests that there are no large-scale differences in methylation levels across these groups.

7. Normalize the data using the function assigned to each student and compare raw data and normalized data. Produce a plot with 6 panels in which, for both raw and normalized data, you show the density plots of beta mean values according to the chemistry of the probes, the density plot of beta standard deviation values according to the chemistry of the probes and the boxplot of beta values. Provide a short comment about the changes you observe. *Optional: do you think that the normalization approach that you used is appropriate considering this specific dataset? Try to color the boxplots according to the group (WT and MUT) and check whether the distribution of methylation values is different between the two groups, before and after normalization ...*

   Our experimental goal is to identify biological variation, but technical variation can hide the real data. The aim of normalization methods is to **remove unavoidable technical variation, in particular systematic bias**.

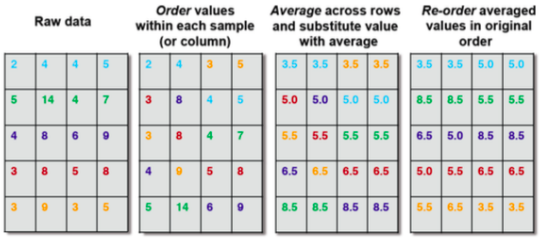
   Everything affects DNA methylation levels. Possibly, this could negatively affect levels in statistical significant manner.

   In order two increase the chance of identifying biological variation, normalisation (not always, but usually) is the starting point and the go to way.

   The easiest way to review the distribution is with the box plot.

   Usually the values that we are interested in are the outliers, lower quartile values and median.

   Quantile normalization → non-parametric normalization procedure in which each feature value (row) is replaced with the mean of the features across all the samples with the same rank or quantile.



It ranks the features (of the rows, so probes) from smallest to largest. it computes the average across all samples for each row, and replace the original value with that average.

> The assumption is that the data distribution of all the samples must be the same. If it's not met we can apply quantile norm but results will be confusing.

But... **Quantile normalization is a global-adjustment method which assumes the statistical distribution of each sample is the same.**

It is not, because there are biological and technical differences among probes.

Type I and Type II probes have a different coverage of CpG rich regions → True biological differences exist

To bypass this normalisation, we could use some derivatives from quantile normalisation, that is **subset quantile normalisation**.

Reference quantiles are estimated on a small set on probes that are considered more reliable and biologically similar and that are used as anchors. Then the intensities of the remaining probes are adjusted by interpolation onto the distribution of the subset probes.**Touleimat.**

- **SWAN (Subset-quantile Within Array Normalization)** (Maksimovic et al., 2012) →> probes are defined as biologically similar on the basis of the CpG content.
- **SQN (preprocessQuantile)** (Touleimat and Tost, 2012) → probes are defined as biologically similar on the basis of the region where they map (based on the 'relation to CpG island' and based on the 'relation to gene sequence'). SQN is applied to all samples simultaneously → at the same time within and between array normalization
- **Normalization using** `preprocessQuantile`**:**
  - The `preprocessQuantile` function is typically used for quantile normalization of microarray data. It ensures that the distributions of beta values across samples are similar after normalization.
  - Now `normalized_wtBeta` and `normalized_mutBeta` contain quantile-normalized beta values for WT and MUT groups, respectively.
- **Comparison of Raw and Normalized Data:**
  - Generate density plots for beta mean and standard deviation values based on the chemistry of the probes for both raw and normalized data.
  - Create boxplots of beta values, optionally colored by group (WT and MUT), to visualize distribution changes.

```
# Subset  the manifest into Type I (dfI) and Type II (dfII) probes based on I
nfinium_Design_Type.
dfI <- Illumina450Manifest_clean[Illumina450Manifest_clean$Infinium_Design_Ty
pe == "I", ]
dfI <- droplevels(dfI)

dfII <- Illumina450Manifest_clean[Illumina450Manifest_clean$Infinium_Design_T
ype == "II", ]
```

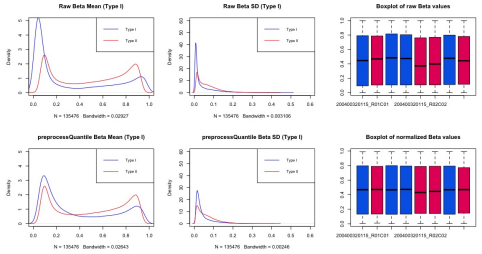
```
dfII <- droplevels(dfII)

#subsetting of the beta matrix according to the chemistry of the probes
beta <- getBeta(MSet.raw)
beta_I <- beta[rownames(beta) %in% dfI$IlmnID, ]
beta_II <- beta[rownames(beta) %in% dfII$IlmnID, ]

# Calculate raw mean and sd
mean_of_beta_I <- apply(beta_I, 1, mean,na.rm=TRUE)
mean_of_beta_II <- apply(beta_II, 1, mean, na.rm=TRUE)
d_mean_of_beta_I <- density(mean_of_beta_I, na.rm = TRUE)
d_mean_of_beta_II <- density(mean_of_beta_II, na.rm = TRUE)

sd_of_beta_I <- apply(beta_I, 1, sd, na.rm = TRUE)
sd_of_beta_II <- apply(beta_II, 1, sd, na.rm = TRUE)
d_sd_of_beta_I <- density(sd_of_beta_I, na.rm = TRUE)
d_sd_of_beta_II <- density(sd_of_beta_II, na.rm = TRUE)


# Normalize using preprocessQuantile
preprocessQuantile_results <- preprocessQuantile(RGset)

# Extract normalized beta values
beta_preprocessQuantile <- getBeta(preprocessQuantile_results)

beta_preprocessQuantile_I <- beta_preprocessQuantile[rownames(beta_preprocess
Quantile) %in% dfI$IlmnID, ]
beta_preprocessQuantile_II <- beta_preprocessQuantile[rownames(beta_preproces
sQuantile) %in% dfII$IlmnID, ]



# Calculate normalized mean and sd
mean_of_beta_preprocessQuantile_I <- apply(beta_preprocessQuantile_I, 1, mea
n,na.rm=TRUE)
mean_of_beta_preprocessQuantile_II <- apply(beta_preprocessQuantile_II, 1, me
an,na.rm=TRUE)
d_mean_of_beta_preprocessQuantile_I <- density(mean_of_beta_preprocessQuantil
e_I, na.rm = TRUE)
d_mean_of_beta_preprocessQuantile_II <- density(mean_of_beta_preprocessQuanti
le_II, na.rm = TRUE)

sd_of_beta_preprocessQuantile_I <- apply(beta_preprocessQuantile_I, 1, sd, n
a.rm = TRUE)
sd_of_beta_preprocessQuantile_II <- apply(beta_preprocessQuantile_II, 1, sd,
na.rm = TRUE)
d_sd_of_beta_preprocessQuantile_I <- density(sd_of_beta_preprocessQuantile_I,
na.rm = TRUE)
d_sd_of_beta_preprocessQuantile_II <- density(sd_of_beta_preprocessQuantile_I
I, na.rm = TRUE)


# Set up the layout for the plots: 2 rows, 3 columns
```

```
par(mfrow = c(2, 3))

### Top Row: Raw Data ###

# Plot densities of mean beta values (raw)
plot(d_mean_of_beta_I, col = "blue", main = "Raw Beta Mean (Type I)", xlim =
c(0, 1), ylim = c(0, 5))
lines(d_mean_of_beta_II, col = "red")
legend("topright", legend = c("Type I", "Type II"), col = c("blue", "red"), l
ty = 1, cex = 0.8)

# Plot densities of standard deviation beta values (raw)
plot(d_sd_of_beta_I, col = "blue", main = "Raw Beta SD (Type I)", xlim = c(0,
0.6), ylim = c(0, 60))
lines(d_sd_of_beta_II, col = "red")
legend("topright", legend = c("Type I", "Type II"), col = c("blue", "red"), l
ty = 1, cex = 0.8)

# Boxplot of raw beta values
targets$Group <- as.factor(targets$Group)
palette(c("#E50068", "#0067E6"))

boxplot(beta, col = targets$Group)
title('Boxplot of raw Beta values')

### Bottom Row: Normalized Data ###

# Plot densities of mean beta values (preprocessQuantile)
plot(d_mean_of_beta_preprocessQuantile_I, col = "blue", main = "preprocessQua
ntile Beta Mean (Type I)", xlim = c(0, 1), ylim = c(0, 5))
lines(d_mean_of_beta_preprocessQuantile_II, col = "red")
legend("topright", legend = c("Type I", "Type II"), col = c("blue", "red"), l
ty = 1, cex = 0.8)

# Plot densities of standard deviation beta values (preprocessQuantile)
plot(d_sd_of_beta_preprocessQuantile_I, col = "blue", main = "preprocessQuant
ile Beta SD (Type I)", xlim = c(0, 0.6), ylim = c(0, 60))
lines(d_sd_of_beta_preprocessQuantile_II, col = "red")
legend("topright", legend = c("Type I", "Type II"), col = c("blue", "red"), l
ty = 1, cex = 0.8)

# Boxplot of normalized beta values
boxplot(beta_preprocessQuantile, col = targets$Group)
title('Boxplot of normalized Beta values')

# Reset par settings to default (1 row, 1 column) to avoid affecting subseque
nt plots
par(mfrow = c(1, 1))
```

- **Density Plots:** After normalization (`preprocessQuantile`), you should observe more similar distributions between WT and MUT groups, indicating successful removal of systematic biases.

- **Boxplots:** Boxplots should show reduced variability and more comparable distributions between groups after normalization.

Appropriateness: Quantile normalization (`preprocessQuantile`) is generally suitable for microarray data like Illumina methylation arrays to ensure comparability across samples. It adjusts each sample's intensity distribution to match the overall distribution, which is essential for downstream statistical analyses.



8. Perform a PCA on the matrix of normalized beta values generated in step 7, after normalization. Comment the plot (do the samples divide according to the group? Do they divide according to the sex of the samples? Do they divide according to the batch, that is the column Sentrix_ID?).

   a. **Normalize the Data**: Assuming you have already performed preprocessQuantile and have `beta_preprocessQuantile` matrix containing normalized beta values.

   ```
   # Assuming beta_preprocessQuantile is your matrix of normalized beta values
   pca_results <- prcomp(t(beta_preprocessQuantile), scale = TRUE)
   ```

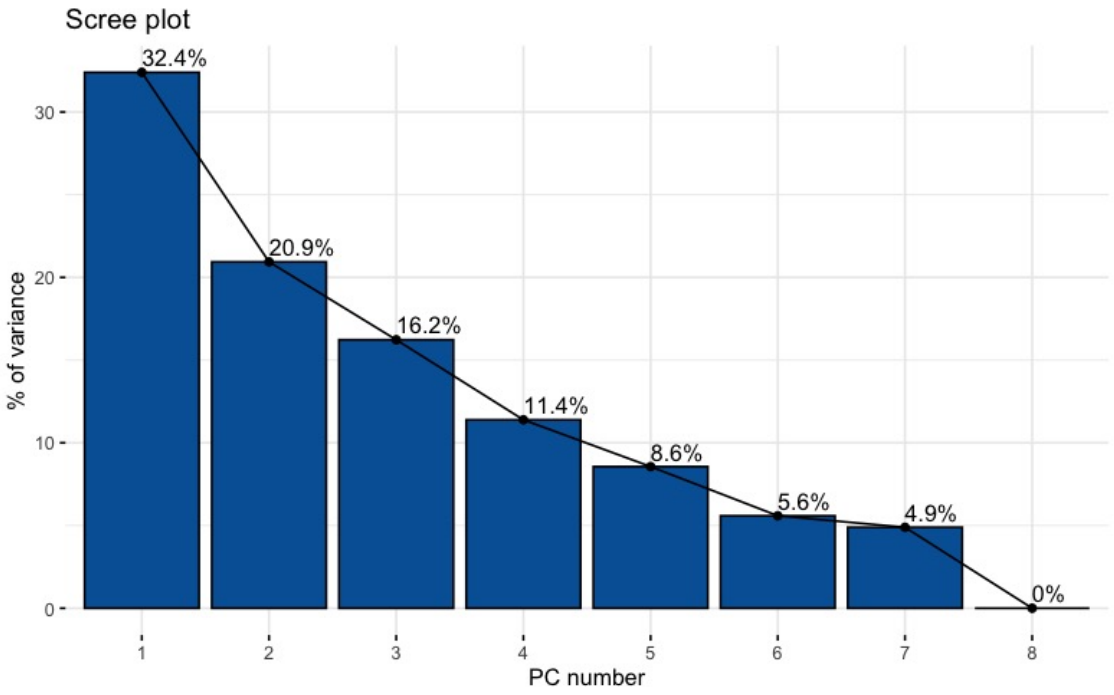
   Here, `prcomp` function is used to perform Principal Component Analysis (PCA) on the transposed matrix (`t(beta_preprocessQuantile)`) to ensure samples are represented as rows and variables (probes) as columns. `scale = TRUE` scales the variables to have unit variance before PCA.

   b. **Visualize Eigenvalues**:

   The eigenvalues provide insights into how much variance each principal component (PC) explains.

   ```
   library(factoextra)
   fviz_eig(pca_results, addlabels = TRUE, xlab = 'PC number', ylab = '% of vari
   ance', barfill = "#0063A6", barcolor = "black")
   ```

   This plot shows the proportion of variance explained by each principal component.

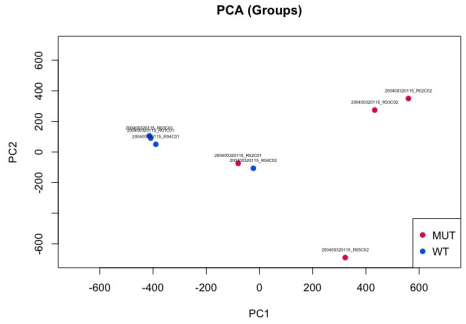


As we can see from the Scree Plot, the **first two PCs** explain the **53.3%** of the variance.

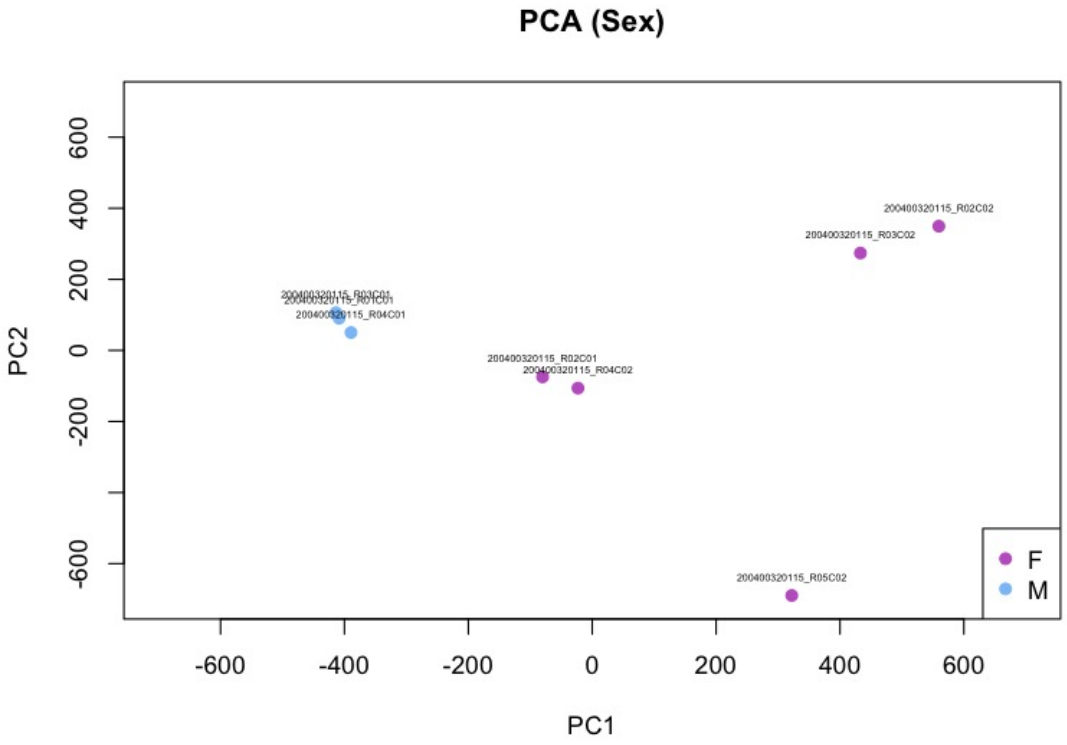
c. **Plot PCA**: Visualize the PCA results to explore how samples cluster based on different factors like group, sex, and batch.

```
targets$Group <- as.factor(targets$Group)
palette(c("#E50068", "#0067E6"))
plot(pca_results$x[, 1], pca_results$x[, 2], cex = 1, pch = 19, col = targets
$Group,
     xlab = "PC1", ylab = "PC2", xlim = c(-700, 700), ylim = c(-700, 700),
     main = 'PCA (Groups)')
text(pca_results$x[, 1], pca_results$x[, 2], labels = rownames(pca_results
$x), cex = 0.4, pos = 3)
legend("bottomright", legend = levels(targets$Group), col = c(1, 2), pch = 1
9, cex = 1.0)


targets$Sex <- as.factor(targets$Sex)
palette(c("#C364CA", "#8BC4F9"))
plot(pca_results$x[, 1], pca_results$x[, 2], cex = 1, pch = 19, col = targets
$Sex,
     xlab = "PC1", ylab = "PC2", xlim = c(-700, 700), ylim = c(-700, 700),
     main = 'PCA (Sex)')
text(pca_results$x[, 1], pca_results$x[, 2], labels = rownames(pca_results
$x), cex = 0.4, pos = 3)
legend("bottomright", legend = levels(targets$Sex), col = c(1:nlevels(targets
$Sex)), pch = 19, cex = 1.0)


targets$Slide <- as.factor(targets$Slide)  # Adjust according to your batch c
olumn name
palette(c("#9e0059", "green"))
plot(pca_results$x[, 1], pca_results$x[, 2], cex = 1, pch = 19, col = targets
```

```
$Slide,
     xlab = "PC1", ylab = "PC2", xlim = c(-700, 700), ylim = c(-700, 700),
     main = 'PCA (Batch)')
text(pca_results$x[, 1], pca_results$x[, 2], labels = rownames(pca_results
$x), cex = 0.4, pos = 3)
legend("bottomright", legend = levels(targets$Slide), col = c(1, 2), pch = 1
9, cex = 1.0)
```

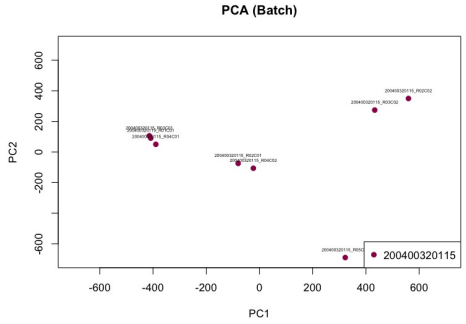


The groups are divided mainly by the PC1, indeed all the WTs are at lower values of PC1 with respect to the MUTs which are mainly located in the right part of the graph, where the PC1 values are higher. However, while the WTs are well clustered according also to PC2, the same is not for the MUTs which are located at different values of PC2, except for R02C02 and R03C02 which cluster very well. Interesting to notice that R02C01, although being a MUT, clusters with the WTs.



By looking at the graph, we can clearly see a division with respect to the sex according to PC1. Moreover, males cluster well together also according to PC2, the same is not for the females. Females

are divided into three clusters, which can be found at different levels of PC2. One of the females cluster is really close to the males cluster, especially according to PC2.

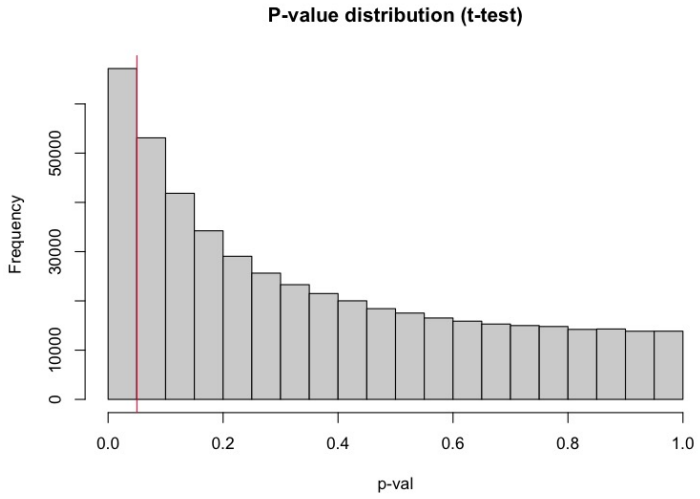


Since all the samples belong to the same batch (**200400320115**) we cannot see a difference in terms of clustering.

- **PCA Plot Interpretation**: PCA is a dimensionality reduction technique that visualizes the variance in data across multiple dimensions (principal components). In the context of DNA methylation data:
  - **Group Division**: Check if samples from different experimental groups (e.g., WT vs MUT) cluster separately in PCA space. Clustering might suggest biological differences related to the experimental condition.
  - **Sex Division**: Explore if samples from different sexes cluster separately. This might indicate sex-specific methylation patterns.
  - **Batch Division**: Investigate if samples processed in different batches or slides cluster together. This can indicate batch effects, which are critical to identify and address in DNA methylation studies.
- **Normalization Impact**: Comment on how preprocessQuantile normalization affects the PCA plot. Look for changes in data structure and clustering patterns compared to raw data. Significant shifts may indicate successful normalization or reveal underlying data quality issues.
- **Biological and Technical Considerations**: Interpret observed clusters in the context of biological hypotheses and technical aspects such as data preprocessing and experimental design. Address any unexpected findings or discrepancies between biological replicates or experimental conditions.

9. a. **Subset Data (Optional)**: If your dataset is large and to speed up the analysis, you can focus on a subset of probes. For example, probes on specific chromosomes like 1, 18, and 21.
   b. **Define Groups**: Assuming `targets` contains information about the groups (WT and MUT).
   c. **Define Differential Methylation Function**: Create a function to perform t-tests for each probe:
   d. **Apply Function Using Parallelization**: Use `future.apply` for parallelization to speed up the process. Ensure you have `future` and `future.apply` packages installed and loaded.
   e. **Combine Results**: Combine the p-values with the probe data into a data frame for further analysis or visualization.
   f. **Visualize P-value Distribution**: Plot the distribution of p-values to assess significance.

```
My_t_test <- function(x) {
  t_test <- t.test(x ~ targets$Group)
  return(t_test$p.value)}

# Apply t-test function across probes (rows)
p_values <- apply(beta_preprocessQuantile, 1, My_t_test)

# Create data frames with all the beta values and the p-value columns
final_ttest <- data.frame(beta_preprocessQuantile, p_values_ttest = p_values)

# Order the probes based on the p-values (from the smallest to the largest valu
e)
final_ttest <- final_ttest[order(final_ttest$p_values_ttest), ]

# Count the number of probes with a p-value <= 0.05
final_ttest_th <- final_ttest[final_ttest$p_values_ttest <= 0.05, ]

hist(final_ttest$p_values_ttest, main = "P-value distribution (t-test)", xlab =
'p-val')
abline(v = 0.05, col = "#ef233c")
> dim(final_ttest_th)
[1] 67180     9
```



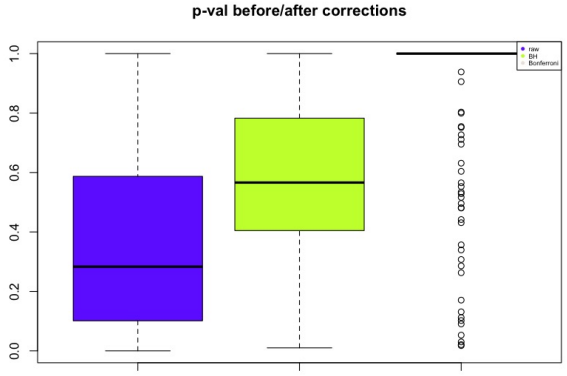
10. Apply multiple test correction and set a significant threshold of 0.05. How many probes do you identify as differentially methylated considering nominal pValues? How many after Bonferroni correction? How many after BH correction?

```
corr_pval_BH <- p.adjust(final_ttest$p_values_ttest,"BH")

corr_pval_bonferroni <- p.adjust(final_ttest$p_values_ttest,"bonferroni")

final_ttest_corr <- data.frame(final_ttest,corr_pval_BH,corr_pval_bonferroni)

#differentially methylated probes according to the threshold (both before and af
```

```
ter the corrections)
before_correction <- nrow(final_ttest_corr[final_ttest_corr$p_values_ttest<= 0.0
5,])

after_Bonferroni <- nrow(final_ttest_corr[final_ttest_corr$corr_pval_bonferroni
<= 0.05,])

after_BH <- nrow(final_ttest_corr[final_ttest_corr$corr_pval_BH <= 0.05,])

diff_meth_df <- data.frame(before_correction, after_Bonferroni, after_BH)
rownames(diff_meth_df) <- c("# differentially methylated probes")
diff_meth_df
```

As expected Bonferroni which is more stringent than BH resulted in only 3 significant probes.

```
# ploltting
par(mfrow=c(1,1))

boxplot(final_ttest_corr [,9:11], col = c("#7038FF", "#C7FF38", "seashell2"),
        names=NA, main='p-val before/after corrections')
legend("topright", legend=c("raw", "BH", "Bonferroni"),
       col=c("#7038FF", "#C7FF38", "seashell2"),pch=19, cex=0.5, xpd=TRUE)
```

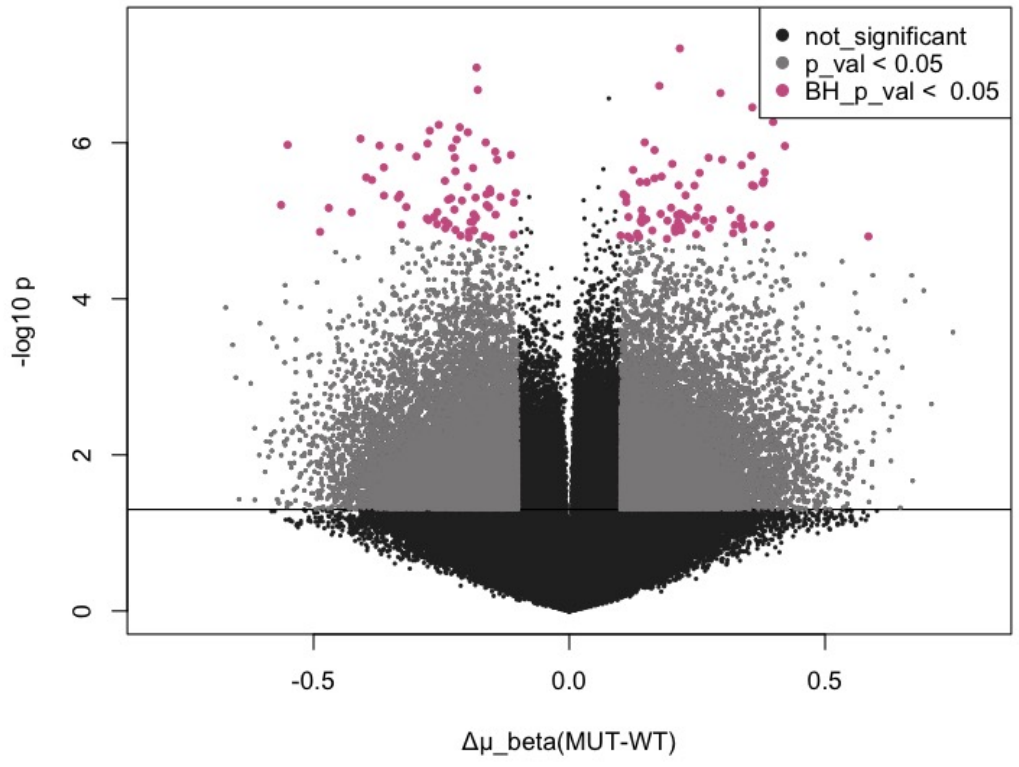


11. Produce a volcano plot and a Manhattan plot of the results of differential methylation analysis

    a. Volcano Plot

```
beta <- final_ttest_corr[,1:8]
beta_groupWT <- beta[,targets$Group=="WT"]
mean_beta_groupWT <- apply(beta_groupWT,1,mean)
beta_groupMUT <- beta[,targets$Group=="MUT"]
mean_beta_groupMUT <- apply(beta_groupMUT,1,mean)

#Now we can calculate the difference between average values:

delta_average <- mean_beta_groupMUT-mean_beta_groupWT

# Now we create a dataframe with two columns, one containing the delta values
and the other with the -log10 of p-values
BH_sig<-final_ttest_corr[10]<0.05
```

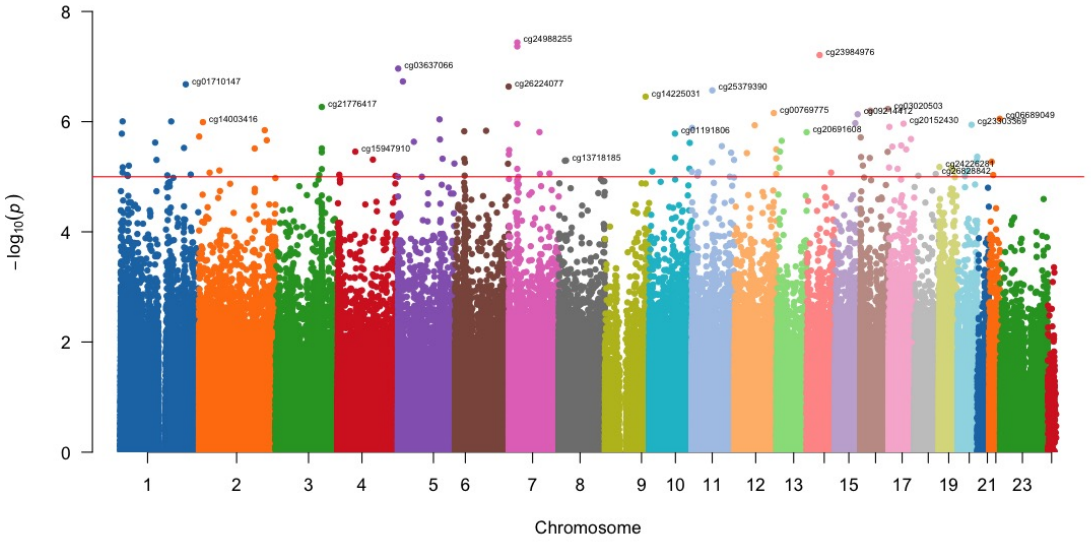
```
toVolcPlot <- data.frame(delta_average,'minus_log10_p_val'= -log10(final_ttes
t_corr$t_test_p_val),BH_sig)

# add a threshold for pvalue significance (for example, 0.01) by using the ab
line function
plot(toVolcPlot[,1], toVolcPlot[,2],pch=16,cex=0.4,col='grey16',
     xlab='Δμ_beta(MUT-WT)',ylab='-log10 p',xlim=c(-0.8,0.8))
abline(h=-log10(0.05))
nominal_sig <- toVolcPlot[abs(toVolcPlot[,1])>0.1 &
                            toVolcPlot[,2]>(-log10(0.05)),]
BH_sig <- toVolcPlot[abs(toVolcPlot[,1])>0.1 & toVolcPlot[,3]==T,]
points(nominal_sig[,1], nominal_sig[,2],pch=16,cex=0.4,col="snow4")
points(BH_sig[,1], BH_sig[,2],pch=19,cex=0.6,col="hotpink3")
legend("topright",
       legend=c("not_significant", "p_val < 0.05", "BH_p_val <  0.05"),
       col=c("grey16", "snow4", "hotpink3"),pch = 19)
```



b. Manhattan Plot

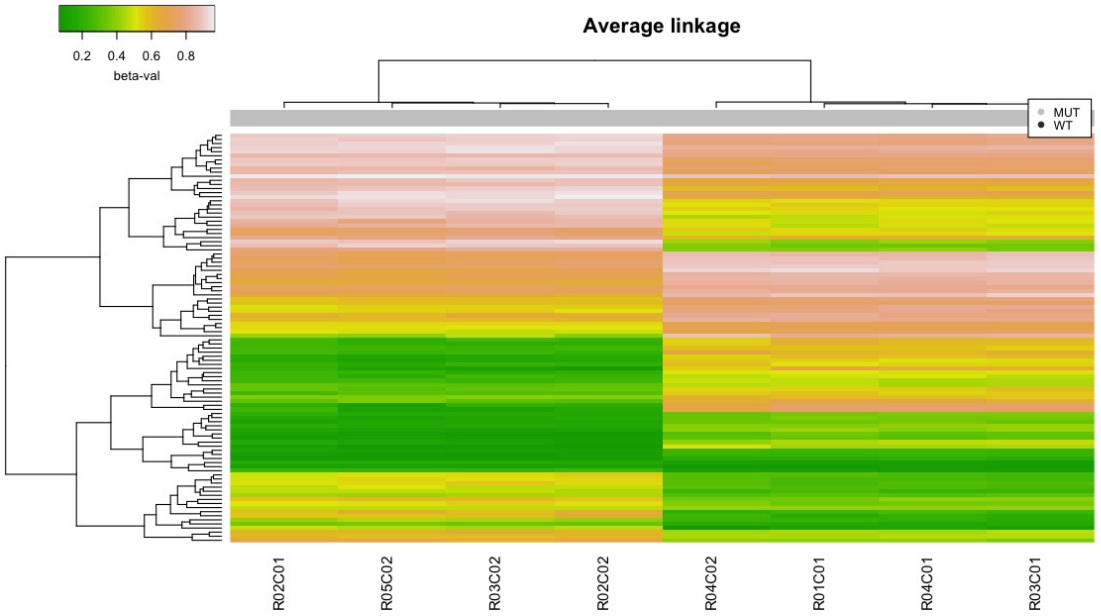
```
### Manhattan Plot
# Prepare the data frame for Manhattan plot
final_ttest_corr_anno <-data.frame(rownames(final_ttest_corr),
                                   final_ttest_corr)
colnames(final_ttest_corr_anno)[1] <- "IlmnID"
```

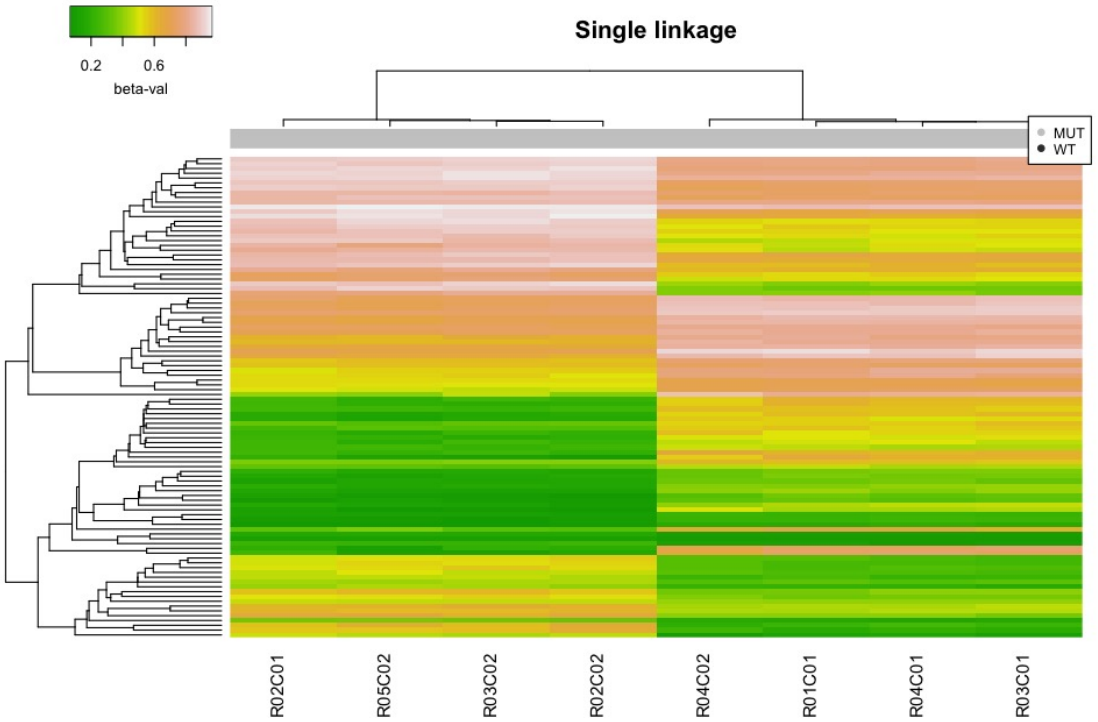
```
final_ttest_corr_anno <- merge(final_ttest_corr_anno, Illumina450Manifest_cle
an,by="IlmnID")
input_Manhattan <- data.frame(ID=final_ttest_corr_anno$IlmnID,
                              CHR=final_ttest_corr_anno$CHR,
                              MAPINFO=final_ttest_corr_anno$MAPINFO,
                              PVAL=final_ttest_corr_anno$p_values_ttest)
head(input_Manhattan)

# Convert chromosome levels for X and Y
levels(input_Manhattan$CHR)[levels(input_Manhattan$CHR) == "X"] <- "23"
levels(input_Manhattan$CHR)[levels(input_Manhattan$CHR) == "Y"] <- "24"
input_Manhattan$CHR <- as.numeric(as.character(input_Manhattan$CHR))

library(qqman)
# Define a color-blind-friendly palette
color_palette <- c("#1F77B4", "#FF7F0E", "#2CA02C", "#D62728", "#9467BD", "#8
C564B", "#E377C2",
                   "#7F7F7F", "#BCBD22", "#17BECF", "#AEC7E8", "#FFBB78", "#9
8DF8A", "#FF9896",
                   "#C5B0D5", "#C49C94", "#F7B6D2", "#C7C7C7", "#DBDB8D", "#9
EDAE5", "#1F77B4",
                   "#FF7F0E", "#2CA02C", "#D62728")

# Plot the Manhattan plot using the defined palette
manhattan(input_Manhattan, snp = "ID", chr = "CHR", bp = "MAPINFO", p = "PVA
L",
          annotatePval = 0.00001, col = color_palette, suggestiveline = FALS
E,
          genomewideline = -log10(0.00001))
```

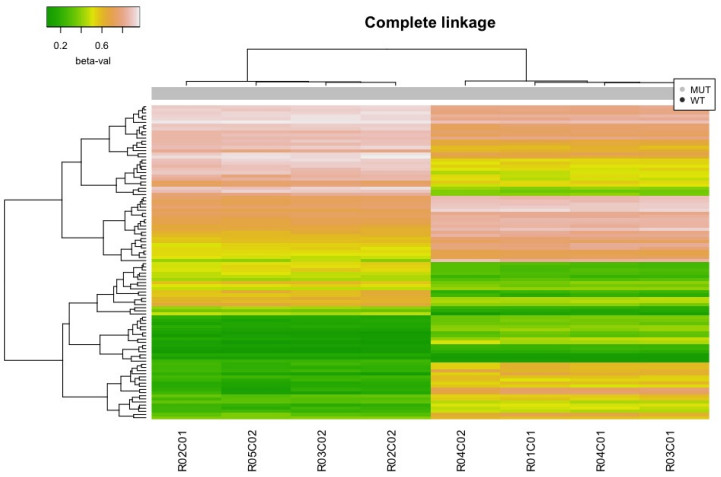


12. Produce an heatmap of the top 100 significant, differentially methylated probes.

```
install.packages("gplots")
library(gplots)

input_heatmap=as.matrix(final_ttest[1:100,1:8])
group_color = c()
i = 1
for (name in colnames(beta)){
  if (name %in% wt){group_color[i]="#393939"}
  else{group_color[i]="#CBCBCB"}
  i = i+1
}
# linkage method: average

heatmap.2(input_heatmap,col=terrain.colors(100),Rowv=T,Colv=T,
          hclustfun = function(x) hclust(x,method = 'average'),
          dendrogram="both",key=T,ColSideColors=group_color,
          density.info="none",trace="none",scale="none",symm=F,
          main="Average linkage",key.xlab='beta-val',key.title=NA,
          keysize=1,labRow=NA)
legend("topright", legend=levels(targets$Group),col=c('#CBCBCB','#393939'),
       pch = 19,cex=0.7)

# linkage method: complete
heatmap.2(input_heatmap,col=terrain.colors(100),Rowv=T,Colv=T,
          dendrogram="both",key=T,ColSideColors=group_color,
          density.info="none",trace="none",scale="none",
          symm=F,main="Complete linkage",key.xlab='beta-val',
          key.title=NA,keysize=1,labRow=NA)
legend("topright", legend=levels(targets$Group),
       col=c('#CBCBCB','#393939'),pch = 19,cex=0.7)

# linkage method: single
heatmap.2(input_heatmap,col=terrain.colors(100),
          Rowv=T,Colv=T,hclustfun = function(x) hclust(x,method = 'single'),
          dendrogram="both",key=T,ColSideColors=group_color,density.info="none",
          trace="none",scale="none",symm=F,main="Single linkage",key.xlab='beta-
val',key.title=NA,keysize=1,labRow=NA)
legend("topright", legend=levels(targets$Group),col=c('#CBCBCB','#393939'),pch =
19,cex=0.7)
```


Average linkage
0.2 0.4 0.6 0.8
beta-val
MUT
WT
R02C01
R05C02
R03C02
R02C02
R04C02
R01C01
R04C01
R03C01


Single linkage
0.2 0.6
beta-val
MUT
WT
R02C01
R05C02
R03C02
R02C02
R04C02
R01C01
R04C01
R03C01


Complete linkage
0.2
0.6
beta-val
MUT
WT
R02C01
R05C02
R03C02
R02C02
R04C02
R01C01
R04C01
R03C01